Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Concurrency

Concurrency

Pemrograman Berorientasi Objek

Concurrency

- Melakukan beberapa aktivitas pada saat yang sama
- Ada 2 unit dasar eksekusi program: proses dan thread
- Di Java concurrency dilakukan di thread

Thread Objects

- Setiap thread diasosiasikan dengan sebuah instansiasi dari class Thread
- 2 strategi penggunaan Thread Object untuk menciptakan aplikasi konkuren
  - Mengendalikan langsung penciptaan dan manajemen thread
  - Melewatkan task aplikasi ke eksekutor

Mendefinisikan Thread

- Menyediakan sebuah Runnable Object

```
public class HelloRunnable implements Runnable {

    public void run() {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[]) {
        (new Thread(new HelloRunnable())).start();
    }

}
```

Mendefinisikan Thread

- Subclass Thread. Class Thread sudah mengimplementasikan Runnable

```
public class HelloThread extends Thread {

    public void run() {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[]) {
        (new HelloThread()).start();
    }

}
```

Mendefinisikan Thread

- Cara yang pertama bisa diterapkan di kelas apapun asal mengimplementasikan interface Runnable
- Cara kedua harus diterapkan pada turunan kelas Thread

Terima Kasih

Concurrency

Thread Feature

Pemrograman Berorientasi Objek

Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Method Sleep

- Menghentikan thread untuk sementara waktu

```
public class SleepMessages {
    public static void main(String args[]) throws InterruptedException {
        String importantInfo[] = {
            "Mares eat oats",
            "Does eat oats",
            "Little lambs eat ivy",
            "A kid will eat ivy too"
        };

        for (int i = 0; i < importantInfo.length; i++) {
            //Pause for 4 seconds
            Thread.sleep(4000);
            //Print a message
            System.out.println(importantInfo[i]);
        }
    }
}
```

Interrupts

- Thread diminta menghentikan eksekusi untuk mengerjakan hal tertentu
- Mengirim interupsi dengan memanggil method interrupt pada thread objek

Menangani Interrupts

- Catch InterruptExecution objek
- Memanggil Thread.interrupted

```
for (int i = 0; i < inputs.length; i++) {
    heavyCrunch(inputs[i]);
    if (Thread.interrupted()) {
        // We've been interrupted: no more crunching.
        return;
    }
}
```

Thread Join

- Method join digunakan untuk menunggu thread lain selesai

```
t.join()
```

Contoh

```
public class SimpleThreads {

    // Display a message, preceded by the name of the current thread
    static void threadMessage(String message) {
        String threadName = Thread.currentThread().getName();
        System.out.format("%s: %s%n", threadName, message);
    }

    private static class MessageLoop implements Runnable {
        public void run() {
            String importantInfo[] = {
                "Mares eat oats",
                "Does eat oats",
                "Little lambs eat ivy",
                "A kid will eat ivy too"
            };
            try {
                for (int i = 0; i < importantInfo.length; i++) {
                    // Pause for 4 seconds
                    Thread.sleep(4000);
                    // Print a message
                    threadMessage(importantInfo[i]);
                }
            } catch (InterruptedException e) {
                threadMessage("I wasn't done!");
            }
        }
    }
```

```
public static void main(String args[]) throws InterruptedException {

    // Delay, in milliseconds before we interrupt MessageLoop
    // thread (default one hour).
    long patience = 1000 * 60 * 60;

    // If command line argument present, gives patience in seconds.
    if (args.length > 0) {
        try {
            patience = Long.parseLong(args[0]) * 1000;
        } catch (NumberFormatException e) {
            System.err.println("Argument must be an integer.");
            System.exit(1);
        }
    }

    threadMessage("Starting MessageLoop thread");
    long startTime = System.currentTimeMillis();
    Thread t = new Thread(new MessageLoop());
    t.start();
```

Contoh

Contoh

```
            threadMessage("Waiting for MessageLoop thread to finish");
            // loop until MessageLoop thread exits
            while (t.isAlive()) {
                threadMessage("Still waiting...");
                // Wait maximum of 1 second for MessageLoop thread to finish.
                t.join(1000);
                if (((System.currentTimeMillis() - startTime) > patience)
                      && t.isAlive()) {
                    threadMessage("Tired of waiting!");
                    t.interrupt();
                    // Shouldn't be long now -- wait indefinitely
                    t.join();
                }
            }
            threadMessage("Finally!");
        }
    }
```

Terima Kasih

Concurrency

Synchronization

Pemrograman Berorientasi Objek

Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Synchronization

- Thread berkomunikasi dengan sharing akses ke field dan object reference fields
- Bisa menimbulkan thread interference dan memory consistency errors
- Thread interference, error yang ditimbulkan ketika sejumlah thread mengakses data bersama (shared)

Synchronization

- Memory consistency errors, error yang diakibatkan view yang tidak konsisten terhadap data bersama (shared)
- Tools yang diperlukan untuk mencegah terjadinya error tersebut adalah sinkronisasi (synchronization)

```
public class Counter {
    private int c = 0;

    public void increment() {
        c++;
    }

    public void decrement() {
        c--;
    }

    public int value() {
        return c;
    }

}
```

Contoh

- Increment:
  - Baca nilai c
  - Tambahkan 1
  - Simpan ke c

Persoalan Thread Interference

- Nilai c=0
- Thread A: Baca nilai c
- Thread B: Baca nilai c
- Thread A: Tambahkan 1
- Thread B: Kurangi 1
- Thread A: Simpan ke c
- Thread B: Simpan ke c

Memory Consistency Errors

- int counter = 0
- Thread A: counter++
- Thread B: System.out.println(counter)

Synchronized Methods

```
public class SynchronizedCounter {
    private int c = 0;

    public synchronized void increment() {
        c++;
    }

    public synchronized void decrement() {
        c--;
    }

    public synchronized int value() {
        return c;
    }
}
```

Synchronized Statements

```
public void addName(String name) {
    synchronized(this) {
        lastName = name;
        nameCount++;
    }
    nameList.add(name);
}
```

Synchronized Statements

```
public class MsLunch {
    private long c1 = 0;
    private long c2 = 0;
    private Object lock1 = new Object();
    private Object lock2 = new Object();

    public void inc1() {
        synchronized(lock1) {
            c1++;
        }
    }

    public void inc2() {
        synchronized(lock2) {
            c2++;
        }
    }
}
```

Liveness

- Kemampuan aplikasi konkuren untuk berjalan dari waktu ke waktu
- Liveness problem: deadlock, starvation, dan livelock
- Deadlock: sebuah situasi dimana dua atau lebih thread terblok selamanya

Liveness

- Starvation: sebuah situasi dimana sebuah thread tidak bisa mendapatkan akses ke resource bersama (shared) dan tidak ada kemajuan
- Livelock: situasi dimana dua atau lebih thread tidak mengalami kemajuan namun tidak terblok

Terima Kasih